Journal of The Korea Institute of Information Security & Cryptology ISSN 1598-3986(Print)
VOL.34, NO.6, Dec. 2024 ISSN 2288-2715(Online)
https://doi.org/10.13089/JKIISC.2024.34.6.1359

IoT 기기 취약점 분석을 위한 펌웨어 추출 및 분석 환경 구축 방법론*

이 예 준,[1†] 이 승 민,[2] 조 효 진[3‡]
[1]중부대학교 (학생), [2,3]연세대학교 (대학원생, 교수)

Firmware Extraction and Analysis Environment Construction Methodology for IoT Devices Vulnerabilities Analysis*

Yejun Lee,[1†] Seungmin Lee,[2] Hyo Jin Jo[3‡]
[1]Joongbu University (Student), [2,3]Yonsei University (Graduate student, Professor)

요 약

최근 몇 년간 급속히 발전하고 있는 IoT 기술과 함께 다양한 IoT 기기가 개발되면서 사용자에게 다양한 기능과 편리함을 제공하고 있다. 하지만 이러한 발전과 동시에 개인정보 유출과 같은 보안 위협에 대한 우려도 커지고 있다. 따라서 IoT 기기의 보안 취약점을 사전에 탐지하는 것은 매우 중요하며, 이를 위해 다양한 취약점 분석 연구가 진행되고 있다. 그러나 IoT 기기의 취약점 분석을 위해 펌웨어를 획득하고 분석 환경을 구축하는 과정은 일반화되기 쉽지 않아 많은 연구자가 어려움을 겪고 있다. 본 논문에서는 IoT 기기 중 사용자가 많이 사용하는 IP 카메라와 같은 소형 IoT 기기에 탑재되는 SOP 타입의 칩을 대상으로 펌웨어를 추출하고 분석 환경을 구축하는 범용적인 방법론을 제안한다. 또한, 상용 IoT 기기에 제안된 방법론을 적용하는 과정을 통한 실장비 분석 환경 구축 사례를 통해 IoT 기기 보안 연구자들에게 펌웨어 추출 및 분석환경 구축에 대한 가이드라인을 제공하고자 한다.

ABSTRACT

In recent years, the rapid development of IoT technology has led to the creation of various IoT devices, offering users a wide range of functionalities and conveniences. However, alongside this advancement, concerns about security threats such as personal information leakage have also grown. Therefore, detecting security vulnerabilities in IoT devices in advance is of utmost importance, and various vulnerability analysis studies are being conducted to address these concerns. However, the process of acquiring firmware and establishing an analysis environment for IoT device vulnerability analysis is not easily standardized, making it challenging for many researchers. In this paper, we propose a universal methodology for extracting firmware and establishing an analysis environment for SOP-type chips embedded in small IoT devices, such as IP cameras, which are widely used by consumers. Furthermore, through the application of the proposed methodology to commercial IoT devices and the process of building an analysis environment for real devices, we aim to provide IoT security researchers with a guideline for firmware extraction and analysis environment setup.

Keywords: IoT Security, Cyber Physical System, Firmware, Embedded, Vulnerability Analysis

Received(10. 15. 2024), Modified(11. 07. 2024), Accepted(11. 19. 2024)
* 본 연구는 한국정보보호학회 산하 IT 플랫폼안정성연구회의 지원으로 수행하였습니다.
† 주저자, exit3015@gmail.com
‡ 교신저자, hyojin.jo@yonsei.ac.kr(Corresponding author)

www.kci.go.kr

I. 서 론

최근 현대 사회에서 IoT(Internet of Things) 기술이 급속히 발전하면서 IP 카메라, 스마트 도어락, 전구, 플러그 등 다양한 종류의 기기가 개발되고 있다. 이러한 기술은 소형 기기뿐만 아니라 냉장고, TV, 로봇청소기와 같은 가전제품에도 접목되어, 사용자에게 일상생활에서의 편리함을 제공한다. 최근 한 보고서[1]에 따르면, 전 세계 IoT 시장 규모와 점유율은 지속적으로 성장하고 있으며, 2033년에는 약 3조 달러의 시장 규모를 달성할 것으로 예측되고 있다.

IoT 기기 사용자의 수요 증가로 인해 새롭게 개발된 IoT 기기는 다양한 공격 벡터와 보안 취약점을 내포하고 있다. MITRE[2]의 조사에 따르면, 2015년부터 현재까지 발견된 IoT 관련 취약점의 수는 약 1,267개에 이르며, 여기에는 DoS(Denial of Service), Sniffing, Spoofing 등의 공격이 포함된다. 이러한 IoT 기기는 취약점이 악용될 경우 봇넷으로 활용되어 2차적인 피해를 초래할 수 있다. 실제로 2016년에 발생한 대표적인 봇넷 공격인 Mirai 봇넷[3,4]은 관리자 계정 설정의 취약점을 통해 IoT 기기에 악성 코드를 전파한 후 이를 기반으로 봇넷을 구성하여 DNS(Domain Name System)를 제공하는 Dyn을 대상으로 대규모 DDoS(Distributed Denial of Service) 공격을 수행했다. 이로 인해 Dyn을 사용하는 트위터, 넷플릭스, 뉴욕타임스 등 1,200개가 넘는 사이트가 일제히 마비되었다. 이와 같이 IoT 기기의 취약점은 사용자의 개인정보 및 프라이버시를 침해할 뿐만 아니라 봇넷으로 활용되어 2차 피해를 일으킬 수 있다.

따라서 IoT 기기의 보안을 강화하기 위해 사전에 취약점을 분석하는 것이 매우 중요하며, 이에 대한 연구도 활발히 진행되고 있다. 취약점 분석을 위한 가장 대표적인 방법은 정적 분석과 동적 분석이다. 이 두 가지 방법 모두 IoT 기기에 대한 펌웨어와 분석 환경 구축이 필요하지만, 정적 분석의 경우, IoT 제조사가 펌웨어를 제공하지 않는 경우가 많으며, 동적 분석의 경우 최근 IoT 기기 제조사들이 UART(Universal Asynchronous Receiver Transmitter), JTAG(Joint Test Action Group)과 같은 디버깅 포트를 숨기는 경우가 많아 연구자들이 취약점 분석 환경 구축에 어려움을 겪고 있다. 또한, 범용적인 가이드라인이 없어 시간과 비용이 많이 소모되고 있다.

따라서 본 논문에서는 IoT 기기 중 많이 사용되는 IP 카메라와 같은 소형 기기에 주로 장착되는 SOP(Small Outline Package) 유형의 칩을 대상으로 펌웨어를 추출하고, 취약점 분석 환경을 구축하는 방법론을 제안한다. 아울러, 실제 IoT 기기를 대상으로 해당 방법론을 적용한 실험 과정과 구축한 분석 환경을 설명함으로써 IoT 기기 취약점 분석 연구자들에게 분석 환경 구축에 관한 가이드라인을 제공하고, 이를 통해 소비자가 많이 사용하는 소형 IoT 기기 취약점 분석을 진행하는 많은 연구자들에게 비용과 시간 절감 효과를 기대할 수 있기를 바란다.

본 논문의 구성은 다음과 같다. 2장에서는 펌웨어 추출 및 취약점 분석과 관련된 기존 연구를 소개하며, 3장에서는 본 논문이 제안하는 방법론을 자세히 설명한다. 4장에서는 상용 IoT 기기 중 하나인 IP 카메라를 대상으로 3장에서 제안한 방법론을 적용한 실험 과정 및 결과를 소개하고, 마지막으로 5장에서는 결론을 제시한다.

II. 관련 연구

본 장에서는 IoT 기기의 펌웨어 추출 및 분석 환경 구축과 관련된 최근 연구들에 대해 소개하고, 이를 통해 연구 동향을 살펴보며 한계점을 인지하고 본 논문이 제안하는 방법론과의 차별점을 서술한다.

2.1 펌웨어 이미지 추출 연구

IoT 기기의 취약점 분석을 위한 펌웨어 이미지 추출 연구는 UART, JTAG과 같은 디버깅 포트를 이용한 방법, 펌웨어 업데이트 과정에서의 패킷 스니핑, 플래시 메모리 덤프 등 다양한 연구가 진행되어 왔다.[5,6] 하지만 최신 IoT 기기에는 제조사가 디버깅 포트를 숨기거나 펌웨어 업데이트 파일에 암호화가 적용되어 기존 연구에서 어려움이 발생했다. 또한 기존 연구 방법론과 같은 경우에는 분석 대상 IoT 기기의 구성 환경에 따라 펌웨어 이미지를 획득하는 작업 흐름이 달라질 수 있어 범용적인 방법론으로 사용이 어려웠다. 본 논문에서는 IoT 기기의 구성 환경에 영향을 받지 않고, 모든 IoT 기기에 동일한 방법으로 펌웨어를 얻을 수 있는 플래시 메모리 칩 오프 방법을 채택했다.

www.kci.go.kr

2.2 동적 분석 환경 구축 연구

IoT 기기의 동적 분석을 위해 펌웨어 이미지를 자동으로 에뮬레이션하는 연구가 진행되었다. Firmadyne[7]는 리눅스 기반 최초의 펌웨어 에뮬레이션 자동화 도구를 제안한 바 있다. 이후, 단순한 휴리스틱 방법을 통해 에뮬레이션 성공률을 크게 향상시킨 FirmAE[8]라는 프레임워크도 제안되었다. 해당 논문에서는 상위 8개 제조업체의 무선 라우터 및 IP 카메라 1,124개의 펌웨어 이미지를 대상으로 Firmadyne과 FirmAE의 성공률을 비교하였다. 그 결과, Firmadyne이 183개(16.28%)의 펌웨어 이미지를 실행한 반면, FirmAE는 892개(79.36%)의 이미지를 성공적으로 에뮬레이션하여 약 4.8배 높은 성공률을 입증하였다.

그러나 이러한 에뮬레이션 연구도 IoT 기기의 모든 의존성과 주변 장치 환경을 설정해주어야 하기 때문에 모든 IoT 기기에 범용적으로 적용하기에는 한계가 있다. 따라서 본 논문에서는 하드웨어 작업을 통해 IoT 기기에 셸 환경을 구성하고, 이를 기반으로 동적 분석 환경을 구축하는 범용적인 방법론을 제시하고자 한다.

III. IoT 기기 취약점 분석 환경 구축

본 장에서는 IoT 기기의 취약점 분석을 위해 펌웨어를 추출하고 분석 환경을 구축하는 방법론에 대해 소개한다. 방법론의 개요는 Fig. 1.과 같으며, 크게 칩 오프, 펌웨어 추출, 펌웨어 변조, 리워크, 테스팅 총 5가지 단계로 구성되어 있다. 각 단계는 IoT 기기에서 펌웨어를 추출하고 분석 환경을 구축하기 위해 반드시 필수적인 과정으로 정상적인 분석 환경 구축을 위해서는 각 단계를 순서대로 건너뛰지 말고 수행해야 한다.

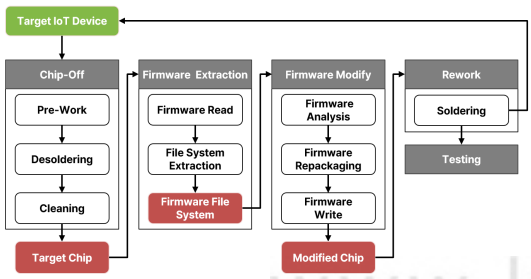


Fig. 1. Overview of Methodology

3.1 Chip-Off

Chip-Off는 PCB(Printed Circuit Board)에 납땜으로 부착된 칩이나 부품을 안전하게 제거하는 작업을 의미한다. 본 논문은 펌웨어를 추출하기 위해 플래시 메모리와 같은 펌웨어가 내장된 칩을 Chip-Off 작업을 통해 탈거한다. Chip-Off 작업에는 납과 플럭스와 같은 유해 물질과 인두기와 같은 고열 장비를 사용하기 때문에 반드시 장갑, 보호구와 같은 안전 장비를 갖추고, 주기적으로 환기가 가능한 환경에서 작업할 것을 권장한다. Chip-Off는 Pre-Work, Desoldering, Cleaning 순서로 진행된다.

3.1.1 Pre-Work

Pre-Work는 Chip-Off를 수행하기 전에 IoT 기기를 사용자 측면에서 연결하여 주요 기능이 정상적으로 작동하는지 확인하고, 여러 가지 연결 벡터를 확인한 뒤 분해를 수행한다. 이때, 추후 기기를 다시 재조립해야 하기 때문에 분해 과정을 상세히 기록하는 것이 좋다. 분해가 모두 완료되면 PCB 기판에서 펌웨어가 내장된 칩을 식별하고, 해당 칩의 데이터시트를 조사하여 확보해야 한다. 특히 펌웨어 추출을 위해 SPI(Serial Peripheral Interface) 통신을 사용하기 때문에 데이터시트에서 칩의 SPI 채널을 확인해 둔다.

3.1.2 Desoldering

Desoldering이란 Chip-Off 단계에서 인두기, 예열기, 열풍기와 같은 고열 도구를 사용하여 부품을 제거하는 작업이다. 본 논문은 IoT 기기의 PCB에서 칩을 제거하기 위해 인두기를 사용했다. 해당 작업은 고열을 사용하므로 칩에 있는 펌웨어가 손상되지 않도록 정교한 작업이 요구되며, 이후 Rework 단계에서 PCB에 다시 칩을 부착해야 하기 때문에 칩뿐만 아니라 PCB 손상에 대해서도 주의를 갖고 작업해야 한다.

인두기로 PCB의 칩을 안전하게 추출하기 위해서는 칩 주변에 플럭스를 도포하고 인두기의 적절한 온도로 열을 가해야 한다. 이때 플럭스는 납의 산화를 방지하여 칩을 부드럽게 떼어 내는 데 도움을 주는 화학 물질로 반드시 필요하다. 인두기의 온도 또한

www.kci.go.kr

칩을 탈거하는 데 중요한 요소로, 온도가 너무 낮으면 납이 제대로 녹지 않아 칩을 떼는 데 어려움을 겪을 수 있으며 반대로, 온도가 너무 높을 경우 칩이나 PCB가 손상되어 펌웨어가 정상적으로 추출되지 않을 수 있다. 따라서 인두기의 적절한 온도값을 설정하는 것은 매우 중요하며, 이는 인두 팁의 상태나 납의 종류에 따라 달라지기 때문에 다양한 실험이 필요하다.

3.1.3 Cleaning

Cleaning은 Desoldering에서 획득한 칩과 PCB에 남아 있는 납과 플럭스를 제거하는 과정이다. 우선 솔더윅을 사용하여 칩과 PCB에 있는 잔여 납을 제거한다. 이때 솔더윅은 납의 흡수를 도와주는 장비로, 납이 있는 부분에 대고 열을 가하면 납이 해당 장비로 흡수된다. 이 과정에서도 마찬가지로 플럭스를 함께 사용하면 납의 산화를 방지하여 잔여 납이 쉽게 제거된다. 잔여 납을 모두 제거하면 이소프로필 알코올과 같은 세척제나 플럭스 제거제를 통해 칩과 PCB에 있는 잔여물을 브러시나 면봉으로 세심하게 닦아낸다. 이 과정을 수행하지 않을 경우, 최종적으로 Rework를 완료한 후 기기를 작동시킬 때 전기적 접촉이 불안정하여 작동이 안 될 수 있으며, 작업 중 남은 여러 불순물로 인해 회로가 단락되어 기기가 고장 날 수 있다. 따라서 Chip-Off 작업을 수행한 뒤에는 반드시 Cleaning 작업을 수행할 것을 권장한다.

3.2 Firmware Extraction

Chip-Off 단계가 완료되면, 획득한 칩에서 펌웨어를 추출하는 작업을 진행해야 한다. SOP 타입의 칩은 SPI 통신을 사용하여 펌웨어 이미지를 읽어올 수 있으며, 이 과정에서 라즈베리 파이, 아두이노 등의 마이크로컨트롤러를 활용한다. 칩의 SPI 채널은 3.1.1에서 확보한 데이터시트를 통해 확인할 수 있다. 펌웨어 추출은 칩에서 펌웨어 이미지를 읽어오는 Firmware Read 과정과 이를 분석하여 파일 시스템을 추출하는 File System Extraction 두 가지 과정으로 구성되어 있다.

3.2.1 Firmware Read

칩에서 펌웨어 이미지를 읽어오기 위해서는 칩의 데이터시트에 맞게 SPI 통신을 구현해야 한다. 따라서 Chip-Off 단계의 3.1.1절에서 확인한 칩의 SPI 채널과 마이크로컨트롤러의 SPI 채널을 핀으로 연결하고, flashrom[9] 오픈소스를 사용하여 칩에서 펌웨어 데이터를 읽어온다. 본 논문이 사용한 flashrom 오픈소스는 다양한 플래시 메모리 칩에서 데이터를 읽고 쓸 수 있는 기능을 제공하며, 수백 가지의 플래시 메모리 칩, 메인보드, 프로그래머를 호환하고 지속적으로 새로운 칩에 대한 지원을 추가하고 있다. 또한, 추출하고자 하는 칩이 오픈소스에 등록되어 있지 않더라도 데이터시트를 기반으로 칩의 정보를 직접 추가하여 사용할 수 있다.

3.2.2 File System Extraction

IoT 기기의 칩에서 획득한 펌웨어 이미지를 분석하여 파일 시스템을 추출하기 위해, 본 논문에서는 binwalk[10]와 firmware-mod-kit[11] 오픈소스를 사용했으며, 두 가지 모두 펌웨어 이미지 내부에서 파일 시스템 및 다양한 데이터 구조를 탐지하고 추출하는 기능을 제공한다. 이때 binwalk의 경우는 파일을 리패키징하는 기능을 제공하지 않아 추가적인 작업이 필요하지만 firmware-mod-kit의 경우 파일 시스템 내부 파일을 수정하고 이를 다시 리패키징하는 기능을 제공하므로 펌웨어 변조 과정에서 사용이 용이하다.

3.3 Firmware Modify

IoT 기기를 동적으로 디버깅하기 위해서는 셸 환경이 구축되어야 한다. 그러나 일반적인 사용자는 셸 환경이 필요하지 않기 때문에 기본적으로 비활성화되어 있다. 따라서 추출된 펌웨어를 변조하여 셸 환경을 활성화하는 작업이 필요하다. 본 절에서는 가장 일반적으로 사용되는 두 가지 펌웨어 변조 지점을 소개하고, IoT 기기에서 추출한 펌웨어 파일 시스템에서 펌웨어를 변조하는 방법과 이를 칩에 다시 덮어쓰는 방법에 대해 설명한다.

www.kci.go.kr

3.3.1 Firmware Analysis

telnetd & /etc/shadow, Fig. 2.의 위쪽과 같이 파일 시스템 내에 telnetd 명령어가 있는 경우, IoT 기기의 부팅 스크립트에서 telnetd를 실행하고 /etc/shadow 파일을 변조하여 장치에 root 권한으로 접속할 수 있다.

nc & backdoor.sh, Fig. 2.의 아래쪽과 같이 파일 시스템 내에 nc 명령어가 있는 경우, 이를 통해 특정 서버로 리버스 셸을 연결할 수 있다. 그러나 nc 명령어를 사용해 부팅 스크립트에서 직접 리버스 셸을 연결하려 할 때, 장치가 네트워크에 연결되지 않은 상태일 수 있다. 따라서 특정 서버에 지속적으로 리버스 셸을 연결하는 스크립트 파일을 작성하고, 이를 부팅 스크립트에서 백그라운드로 실행한다.

앞서 두 가지 방법에 해당하는 명령어가 파일 시스템 내부에 없는 경우, IoT 기기의 아키텍처를 확인하고 busybox, socat, nc와 같은 명령어를 정적으로 컴파일하여 분석자가 원하는 명령어를 추가할 수 있다. 그러나 이러한 명령어 삽입은 펌웨어 용량을 증가시켜 리패키징 과정에서 한계점이 존재할 수 있다.

3.3.2 Firmware Repackaging

펌웨어 리패키징은 수정된 파일 시스템을 다시 펌웨어로 패킹하는 과정으로, 펌웨어를 변조했다면 반드시 수행해야 하는 단계이다. 이 과정에서는 파일 시스템을 추출할 때 사용한 오픈소스 도구에 따라 적절한 방법을 사용해야 한다. 예를 들어, binwalk 도구를 사용하여 파일 시스템을 추출했다면, 파일 시스템의 유형에 맞게 리패키징해야 한다. 만약 파일 시스템이 squashfs인 경우, mksquashfs 명령어를 사용하여 파일 시스템을 다시 패킹하며, 이때 기존 파일 시스템의 압축 알고리즘을 동일하게 사용해야 한다. squashfs가 아닌 경우에는 mkfs.〔파일시스템 유형〕 명령어를 사용해 패킹할 수 있다. 또한, firmware-mod-kit 도구를 사용하여 파일 시스템을 추출한 경우, fmk 디렉터리가 생성되었을 것이다. 수정할 부분을 fmk/rootfs 안에서 변경한 후 /firmware-mod-kit/build-firmware.sh 스크립트를 통해 리패키징한다. firmware-mod-kit 도구는 파일 시스템 추출 시 전체 펌웨어 구조를 저장해 두기 때문에, 펌웨어를 쉽게 변조할 수 있다.

3.3.3 Firmware Write

3.2.1절에서 설명한 펌웨어 읽기 과정과 동일하게, 칩의 SPI 핀을 마이크로컨트롤러와 매핑하여 연결한 후, flashrom 도구를 사용하여 칩을 인식시키고 변조된 펌웨어를 칩에 덮어쓴다.

3.4 Rework

Rework는 앞서 떼어낸 칩을 다시 PCB에 솔더링(soldering)하는 과정이다. 이 과정을 진행하기 전에 칩과 PCB가 모두 깨끗하게 세척되었는지 확인해야 하며, 칩의 타입에 맞는 적절한 장비를 사용하여 정밀하게 작업해야 한다. 또한, 납의 산화를 방지하고 부드러운 솔더링을 위해 플럭스를 사용하는 것이 좋다. 칩이 잘 부착되었다면, PCB 주변의 잔여 불순물을 이소프로필 알코올로 반드시 세척해야 한다.

3.5 Testing

최종적으로 솔더링이 완료되면, 3.1.1절에서 기록해 둔 분해 과정을 역순으로 재조립을 진행한다. 이후 IoT 기기를 부팅시켜 정상적으로 기능을 수행하는지 확인하고, 분석자가 변조한 펌웨어 포인트가 제대로 적용되었는지도 점검한다. 만약 기기가 정상적으로 부팅되지 않는다면, 하드웨어 작업 중 칩이나 PCB가 손상되었거나 펌웨어 변조 과정에서 문제가 발생했을 가능성이 있다. 이 경우에는 동일한 IoT 기기와 비교하여 문제의 원인을 식별하고 해결해야 한다.

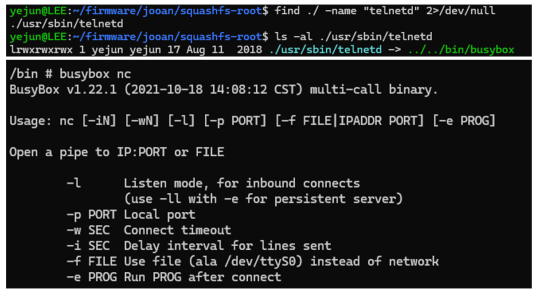


Fig. 2. Check the command ((up) telnetd /(down) netcat)

www.kci.go.kr

IV. Case Study: JOOAN IP Camera

본 장에서는 실제 상용 IoT 기기를 대상으로 3장에서 제안한 방법론을 적용한 실험 과정 및 결과를 서술한다. 본 논문에서 실험한 장비는 중국 브랜드 JOOAN의 IP 카메라로, IoT 기기 중 많이 사용되는 카메라를 대상으로 선정하였다.

4.1 Chip-Off

Pre-Work, 우선 본격적인 하드웨어 작업에 앞서 장치를 사용자 측면에서 연결하여 정상적인 장치 기능을 확인한다. 실험 장비의 경우 Fig. 3.의 왼쪽과 같이 cam720이라는 애플리케이션을 통해 실시간 영상을 제공하며, 웹 서비스를 통해 장치 설정을 변경할 수 있다. 이후 장치를 분해하여 Fig. 3.의 오른쪽과 같이 PCB에서 추출 대상 칩인 XM25QG64C 플래시 메모리를 식별하였다. 또한 추후 펌웨어 추출 단계에서 SPI 채널을 사용하기 위해 XM25QG64C의 데이터시트를 조사하여 미리 확보하고, Fig. 4.와 같이 SPI 채널과 마이크로컨트롤러의 핀을 매핑한 테이블을 정리하였다.

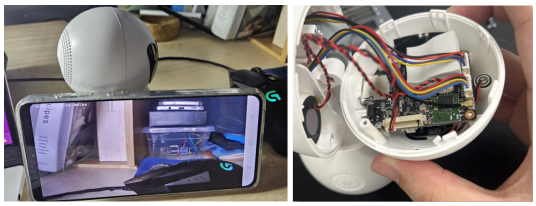
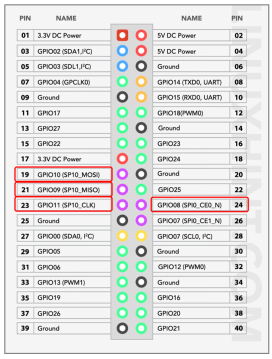


Fig. 3. Pre-work (Verification of normal device operation (left) / Device disassembly and chip identification (right))



PIN	NAME	NAME	PIN
01	3.3V DC Power	5V DC Power	02
03	GPIO02 (SDA1,I²C)	5V DC Power	04
05	GPIO03 (SDL1,I²C)	Ground	06
07	GPIO04 (GPCLK0)	GPIO14 (TXD0, UART)	08
09	Ground	GPIO15 (RXD0, UART)	10
11	GPIO17	GPIO18(PWM0)	12
13	GPIO27	Ground	14
15	GPIO22	GPIO23	16
17	3.3V DC Power	GPIO24	18
19	GPIO10 (SPI0_MOSI)	Ground	20
21	GPIO09 (SPI0_MISO)	GPIO25	22
23	GPIO11 (SPI0_CLK)	GPIO08 (SPI0_CE0_N)	24
25	Ground	GPIO07 (SPI0_CE1_N)	26
27	GPIO00 (SDA0, I²C)	GPIO07 (SCL0, I²C)	28
29	GPIO05	Ground	30
31	GPIO06	GPIO12 (PWM0)	32
33	GPIO13 (PWM1)	Ground	34
35	GPIO19	GPIO16	36
37	GPIO26	GPIO20	38
39	Ground	GPIO21	40



추출할 칩	Raspberry Pi 4
CS (1)	SPI_CE0_N (24)
DO (MISO) (2)	SPI0_MISO (21)
VSS (GND) (4)	GROUND (9)
VCC (8)	3.3V DC Power (1)
CLK (6)	SPI0_CLK (23)
DI(MOSI) (5)	SPI0_MOSI (19)

Fig. 4. Pre-work (Obtaining data sheets and mapping SPI channels)

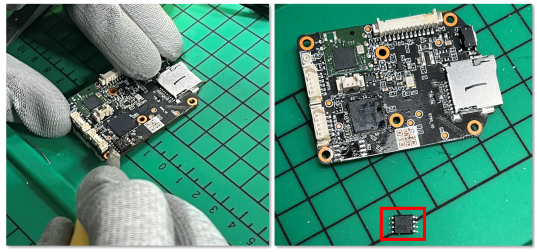


Fig. 5. Desoldering process (left) / Chip removal complete (right)

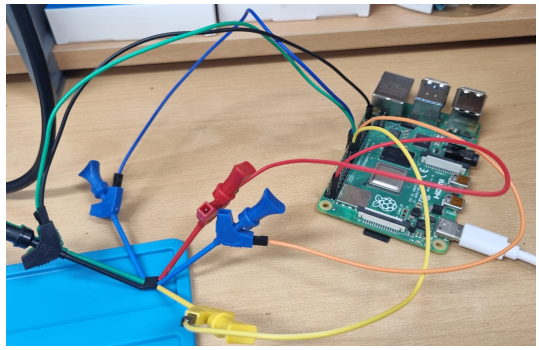


Fig. 6. Firmware Read (Connect SPI pins of target chip and Raspberry Pi 4)

Desoldering, 실험 장비의 플래시 메모리는 SOP 타입으로, 인두기를 사용하여 Chip-Off를 수행한다. 본 논문에서는 인두기 온도를 380°C로 설정하였으며, 칩의 다리 부분에 열을 가하며 칩을 탈거하였다. Fig. 5.는 실제 장비에서 칩을 탈거하는 과정과 추출한 칩의 사진이다.

Cleaning, Chip-Off 이후 솔더윅을 통해 PCB에 남아있는 잔여 납을 제거하고 이소프로필 알코올과 플럭스 제거제를 통해 남아있는 플럭스와 잔여물을 깨끗이 세척하여 준다.

4.2 Firmware Extraction

Firmware Read, Chip-Off를 정상적으로 마친 뒤, Cleaning이 완료된 칩을 미리 확보한 데이터시트와 SPI 채널 매핑 테이블을 참고하여 Fig. 6과 같이 라즈베리파이 4와 연결한 뒤 flashrom을 사용하여 펌웨어를 읽어온다. 이때 Chip-Off를 하지 않고 PCB 위에서 칩을 연결하여 펌웨어 추출이 가능한 경우도 있지만, 추출 과정에서 PCB의 노이즈로 인해 완전한 펌웨어가 추출되지 않을 수 있다. 따라서 Chip-Off 과정을 수행하여 탈거된 칩에서 펌웨

www.kci.go.kr

어를 추출하는 것을 권장한다.

File System Extraction, 펌웨어 파일을 읽어온 뒤 binwalk 도구를 사용하여 파일 시스템을 추출할 수 있다. 실험 장비의 경우 두 개의 squashfs가 존재했다.

4.3 Firmware Modify

Firmware Analysis, 실험 장비인 JOOAN IP 카메라의 경우 busybox를 통해 여러 명령어를 사용할 수 있다. 본 논문은 그중 telnetd 명령어를 사용하여 Fig. 7.과 같이 부팅 스크립트에서 명령어를 실행하도록 수정하고, shadow 파일을 수정하여 장치에 telnet 접속이 가능하도록 펌웨어를 변조하였다.

Firmware Repackaging, 파일 시스템 변조가 완료된 후 mksquashfs 명령어를 통해 squashfs로 리패키징한다. 이후 binwalk에서 확인된 기존의 squashfs 영역에 HxD 도구를 사용하여 Fig. 8.과 같이 펌웨어 이미지를 덮어쓴다. 이때 변조된 squashfs의 크기가 펌웨어 이미지의 다른 파티션을

```
 96 # Run start script
 97 if [ -f /mnt/mtd/startapp ]; then
 98         echo "==> startapp ok"
 99     /mnt/mtd/startapp &
100 fi
101
102 /usr/sbin/telnetd &
```

```
1 root::10933:0:99999:7:::
```

Fig. 7. Firmware Modify ((up)Boot Script / (down) shadow)

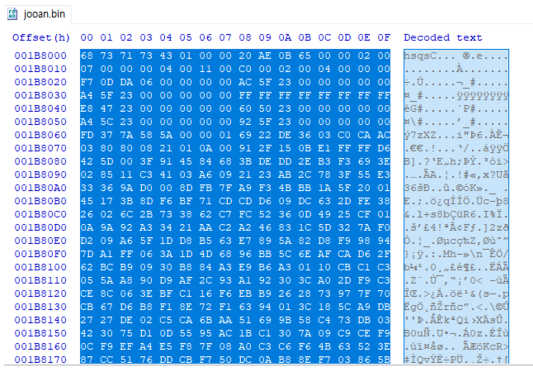


Fig. 8. Firmware Repackaging (Firmware Overwrite via Hex Editor)

넘어서지 않아야 한다. Jooan IP Camera의 경우 두 번째 squashfs 파티션이 0x418000 부터 시작하기 때문에 0x1b8000 부터 0x260000 크기 안으로 변조되어야 한다.

Firmware Write, 앞선 4.2절의 과정과 동일하게 칩을 라즈베리파이 4와 연결하고 flashrom을 통해 칩에 변조된 펌웨어를 쓸 수 있다.

4.4 Rework

변조된 플래시 메모리를 인두기를 통해 PCB 기판에 다시 부착한다. 앞서 칩이 있던 자리의 잔여 납을 모두 제거했기 때문에, 칩의 부착 위치에 납을 다시 올려 솔더링을 진행한다. Fig. 9.와 같이 칩이 다시 정상적으로 부착되면 마무리로 플럭스와 잔여물을 Cleaning 작업을 통해 제거해 준다.

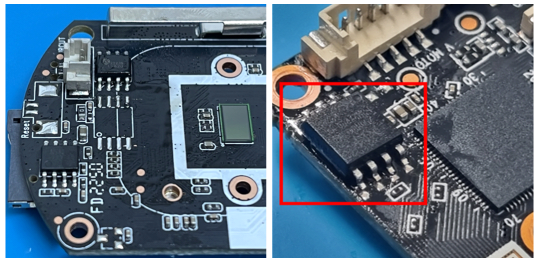


Fig. 9. Rework (Lead attachment via presoldering (left) / Chip attached via soldering (right))

4.5 Testing

최종적으로 Rework 작업이 완료되면 장치의 PCB를 분해의 역순으로 재조립하고 전원을 연결한다. 부팅이 완료되면 변조된 펌웨어로 인해 telnet으로 IoT 기기의 root 셸 접속이 가능하다. 또한 JOOAN IP 카메라의 경우 영상 저장을 위해 SD 카드를 /mnt/sd_card 경로로 마운트하기 때문에 SD 카드에 장비 아키텍처에 맞는 gdbserver를 삽입하고 이를 root 셸에서 실행하면 Fig. 10.과 같이 동적 분석 환경을 구축할 수 있다.

V. 한계점

본 장에서는 마지막으로 본 논문이 제안한 방법론의 한계점에 대해 서술한다. IoT 산업이 발전하면서

www.kci.go.kr



Fig. 10. Testing (Building an analysis environment by running gdbserver)

IoT 보안 기술에 대해서도 많은 연구와 관심이 지속적으로 확대되어 왔다. 이에 따라 펌웨어 무결성과 신뢰성을 보장하는 시큐어 부트 기술에 대한 많은 연구가 진행되었으며, IoT 장치를 보호하는데 효과적으로 사용되고 있다.[11,12,13] 시큐어 부트가 적용된 IoT 기기에서는 본 논문이 제안한 방법론만으로는 분석 환경을 구축할 수 없다. 이 경우에는 IoT 기기에 결함을 주입하는 글리칭 공격 기법을 활용하여 본 논문에서 제안하는 방법론을 확대할 수 있다.[14,15] 그러나 IoT 기기를 제조하는 많은 업체의 가격 경쟁이 심화되면서, 대부분의 저가형 IoT 기기에서는 비용이 증가하는 시큐어 부트 기술을 적용하지 않고 있다. 따라서, 공유기, IP 카메라와 같이 소비자에게 많이 노출되어 있는 저가형 IoT 기기에서는 본 논문에서 제안하는 방법론을 적용할 수 있음을 실험을 통해 증명하였다.

VI. 결 론

본 논문에서는 IoT 기기 취약점 분석을 위해 IoT 기기에서 펌웨어를 추출하고 동적 분석 환경을 구축하는 체계적인 방법론을 제시하고 이를 증명하였다. 특히, 본 연구는 일상 속에서 널리 사용되는 저가형 IoT 기기에 주로 탑재된 SOP 칩을 중심으로 수행되어, 범용성과 실용성을 함께 갖추고 있다. 따라서 IoT 보안 연구자들이 보다 빠르고 효율적으로 취약점 분석 환경을 구축할 수 있도록 기여한다. 향후 연구에서는 본 논문이 제안한 방법론을 시큐어 부트가 적용된 고가형 IoT 기기까지 확장하여 보안이 고도화된 장치에 관한 취약점 연구 가능성을 높일 필요가 있다. 또한, 본 논문에서 제안한 펌웨어를 변조 과정의 자동화를 통해 연구자들이 더욱 쉽게 변조 지점을 찾고 적용할 수 있는 환경을 마련하는 것이 중요하다. 본 연구를 통해 IoT 보안 분야의 발전에 기여하고, 더 안전한 IoT 생태계 구축에 도움이 되기를 기대한다.

References

[1] Precedence Research, "Internet of Things (IoT) Market Size, Share, and Trends 2024 to 2033" https://www.precedenceresearch.com/internet-of-things-market, accessed Nov. 2024.

[2] MITRE, "CVE Records for IoT Vulnerabilities" https://cve.mitre.org/cgi-bin/cvekey.cgi?keyword=IoT, accessed Oct. 2024.

[3] C. Kolias, G. Kambourakis, and A. Stavrou, "DDoS in the IoT: Mirai and other botnets," IEEE Computer, vol. 50, no. 7, pp. 80-84, Jul. 2017.

[4] M. Antonakakis, T. April, and M. Bailey, "Understanding the Mirai botnet," Proceedings of the 26th USENIX Security Symposium, pp. 1093-1110, Aug. 2017.

[5] Eul-gyu Im, "Firmware Analysis of Embedded Devices," Journal of Korean Institute of Information Scientists and Engineers, 34(1), pp. 16-21, Jan. 2016.

[6] S.U. Haq, Y. Singh, and A. Sharma, "A survey on IoT & embedded device firmware security: architecture, extraction techniques, and vulnerability analysis frameworks," Discover Internet of Things, vol. 3, no. 17, pp. 1-18, Oct. 2023.

www.kci.go.kr

〔7〕 D.D. Chen, M. Egele, and M. Woo, "Towards Automated Dynamic Analysis for Linux-based Embedded Firmware," Proceedings of the Network and Distributed System Security Symposium, pp. 1-10, Jan. 2016.

〔8〕 Mingeun Kim, Dongkwan Kim, and Eunsoo Kim, "FirmAE: Towards Large-Scale Emulation of IoT Firmware for Dynamic Analysis," Proceedings of the 36th Annual Computer Security Applications Conference, pp. 733-745, Dec. 2020.

〔9〕 Github, "Flashrom" https://github.com/flashrom/flashrom, accessed Nov. 2024.

〔10〕 Github, "Binwalk" https://github.com/ReFirmLabs/binwalk, accessed Nov. 2024.

〔11〕 Github, "Firmware-Mod-Kit" https://github.com/rampageX/firmware-mod-kit, accessed Nov. 2024.

〔12〕 A. Ukil, J. Sen, and S. Koilakonda, "Embedded security for Internet of Things," Proceedings of the 2011 2nd National Conference on Emerging Trends and Applications in Computer Science, pp. 1-6, Mar. 2011.

〔13〕 Siva Bala Chandar M, and Arunkumar S, "Implementation and Verification of Secure Boot in Embedded Systems," Proceedings of the 2024 International Conference on Advances in Modern Age Technologies for Health and Engineering Science, pp. 1-4, May. 2024.

〔14〕 C. Profentzas, M. Gunes, and Y. Nikolakopoulos, "Performance of secure boot in embedded systems," Proceedings of the 2019 15th International Conference on Distributed Computing in Sensor Systems, pp. 198-204, May. 2019.

〔15〕 B. Yuce, P. Schaumont, and M. Witteman, "Fault attacks on secure embedded software: Threats, design, and evaluation," Journal of Hardware and Systems Security, vol. 2, pp. 111-130, May. 2018.

〔16〕 A. Cui and R. Housley, "BADFET: Defeating modern secure boot using Second-Order pulsed electromagnetic fault injection," Proceedings of the 11th USENIX Conference on Offensive Technologies, pp. 3, Aug. 2017.

www.kci.go.kr

〈저 자 소 개〉



이 예 준 (Yejun Lee) 학생회원
2019년 3월~현재: 중부대학교 정보보호학과 학사과정
〈관심분야〉 IoT 보안, 시스템 보안



이 승 민 (Seungmin Lee) 학생회원
2024년 2월: 숭실대학교 소프트웨어학부 졸업
2024년 3월~현재: 연세대학교 정보대학원 석사과정
〈관심분야〉 IoT/CPS 보안, 자동차 보안



조 효 진 (Hyo Jin Jo) 종신회원
2009년 2월: 고려대학교 산업공학과 졸업
2016년 2월: 고려대학교 정보보호대학원 박사 졸업
2018년 8월: University of Pennsylvania 박사 후 연구원
2020년 8월: 한림대학교 정보과학대학 조교수
2024년 2월: 숭실대학교 소프트웨어학부 조교수
2024년 3월~현재: 연세대학교 정보대학원 부교수
〈관심분야〉 자동차 보안, IoT/CPS 보안, 프라이버시

www.kci.go.kr